

Troubleshooting

Erros que acontecem de verdade, e a saída de cada um. Os da parte de infraestrutura vieram do histórico real de deploy do `seem-backend`.

Ambiente

`mise: command not found`

A linha de `activate` não foi para o arquivo certo. Confira o fim do `~/.bashrc` (ou `~/.zshrc`) e rode `exec bash`.

A instalação do Ruby falha compilando

Faltou dependência de build. Rode o bloco de `apt install` / `brew install` de `00-preparacao.md` e tente `mise install ruby@3.4.9` de novo.

`gem install rails` reclama de permissão

Você está usando o Ruby do sistema, não o do mise. Confira:

```
which ruby # tem que apontar para dentro de ~/.local/share/mise
```

`permission denied` ao rodar `docker`

Você não está no grupo `docker`:

```
sudo usermod -aG docker $USER
```

E **saia e entre de novo na sessão**. Reabrir o terminal não basta.

No Windows, `docker` não aparece dentro do Ubuntu

Docker Desktop → Settings → Resources → WSL Integration → habilite a distro `Ubuntu-24.04`.

Banco

`address already in use` ao subir o compose

Já existe um Postgres na sua máquina ocupando a 5432.

```
DB_PORT=5433 docker compose up -d
export DB_PORT=5433          # e exporte no shell onde roda o Rails
```

Para achar quem ocupa: `ss -ltnp | grep 5432`.

`PG::ConnectionBad: could not connect to server`

Nesta ordem:

```
docker compose ps          # o container está "healthy"?
docker compose logs db     # o que ele diz?
echo $DB_PORT              # bate com a porta publicada?
```

`PG::UndefinedTable` ou `PendingMigrationError`

Faltou migrar:

```
bin/rails db:migrate
```

Quero começar do zero

```
bin/rails db:reset          # apaga, recria, carrega o schema e roda os seeds
```

Testes

Um teste passa sozinho e falha junto com os outros

Vazamento de estado entre testes. Suspeitos, em ordem:

1. um `Rails.cache` compartilhado (o `test_helper` troca por `MemoryStore` a cada teste);
2. `ActionMailer::Base.deliveries` não limpo;
3. dependência de ordem: o Minitest embaralha de propósito, e isso é uma qualidade.

Rode com a mesma semente para reproduzir: `bin/rails test --seed 1234`.

O teste de logout passa mas a revogação não funciona

Em teste, o `Rails.cache` padrão é o `null_store`: não guarda nada, e `revoked?` sempre devolve `false`. O `test_helper.rb` troca por `MemoryStore` justamente por isso. Se você recriou o arquivo, recoloque o `setup`.

`Bullet::Notification::UnoptimizedQueryError`

Consulta N+1: você carregou uma coleção e acessou uma associação item a item. Adicione `includes`. (O `seem-backend` usa o Bullet; o app do curso não.)

Autenticação

Sempre 401, mesmo com o token certo

Cheque, nesta ordem:

1. o cabeçalho é exatamente `Authorization: Bearer <token>`, com o espaço;
2. o token não foi revogado (você fez logout com ele?);
3. o `token_version` do banco bate com o `ver` do token: trocar a senha incrementa a versão;
4. o `JWT_SECRET` é o mesmo que emitiu o token. Reiniciar o app em dev sem `JWT_SECRET` definido usa o `secret_key_base`, que é estável, mas em produção um segredo diferente invalida tudo.

Decodifique o token em jwt.io e olhe o `exp` e o `ver`.

O e-mail não chega em desenvolvimento

Ele não é enviado: o `letter_opener` abre no navegador. Se você está rodando `curl` e não vendo nada, o arquivo está em `tmp/letter_opener/`.

`ActionController::ParameterMissing (400)`

Faltou um campo obrigatório no corpo. O `params.expect` exige todas as chaves declaradas. Confira o JSON e o `Content-Type: application/json`.

422 no cadastro sem dizer o motivo direito

Olhe `error.details` na resposta: cada item tem `field` e `message`.

GitHub

O CI ficou vermelho logo no primeiro push

O `rails new` já cria um `.github/workflows/ci.yml`, e ele roda sozinho a cada push. Nas Aulas 1 e 2 ele deve passar. Se ficar vermelho, abra o log em **Actions** e veja qual dos três jobs quebrou:

- **lint**: é o RuboCop. Rode `bin/rubocop -a` para corrigir o que dá sozinho.
- **test**: rode `bin/rails test` na sua máquina; o erro é o mesmo.
- **scan_ruby**: Brakeman ou uma gem com CVE. `bin/brakeman --no-pager` mostra o motivo.

Na Aula 4 esse arquivo é substituído pelo nosso, que roda os testes contra um Postgres de verdade.

`gh: command not found`

O GitHub CLI não está instalado. Volte ao passo 7 de `00-preparacao.md`.

`gh repo create` reclama de autenticação

```
gh auth status      # tem que dizer "Logged in to github.com"
gh auth login        # se não estiver
```

O SSH da sua máquina

Na Aula 4 o servidor é a sua própria máquina, e o Kamal chega nela por SSH.

`Connection refused` na porta 22

O `sshd` não está rodando.

```
sudo service ssh start      # Ubuntu, Debian, WSL2
sudo systemctl start sshd   # Fedora
```

No macOS, ligue em **Ajustes do Sistema** → **Geral** → **Compartilhamento** → **Sessão remota**.

Confira que ele está escutando:

```
ss -tlnp | grep :22      # Linux e WSL2
sudo lsof -i :22         # macOS
```

Funcionava, reiniciei o Windows, e parou

É o WSL2: sem o systemd habilitado, ele não sobe o `sshd` no boot.

```
sudo service ssh start
```

Para não repetir isso toda vez, habilite o systemd em `/etc/wsl.conf`:

```
[boot]
systemd=true
```

E depois, no PowerShell: `wsl --shutdown`.

Permission denied (publickey)

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/capacita ~/.ssh/authorized_keys
ssh -i ~/.ssh/capacita -o IdentitiesOnly=yes "$USER"@127.0.0.1
```

O SSH **recusa** usar chave com permissão frouxa, e recusa `authorized_keys` que outros possam escrever. Se persistir, confira que a chave pública realmente entrou:

```
cat ~/.ssh/authorized_keys
```

Too many authentication failures

O SSH ofereceu todas as chaves do seu agente e o servidor cortou antes de chegar na certa.

```
ssh -i ~/.ssh/capacita -o IdentitiesOnly=yes "$USER"@127.0.0.1
```

Host key verification failed

A chave do host mudou (reinstalou o `sshd`, ou trocou de máquina).

```
ssh-keygen -R 127.0.0.1
```

docker: command not found só pela SSH

Uma sessão SSH não interativa carrega um `PATH` menor que o seu terminal. Confira:

```
ssh -i ~/.ssh/capacita "$USER"@127.0.0.1 'which -a docker'
```

Se não achar, o Docker foi instalado de um jeito que só existe no seu shell interativo (Docker Desktop com integração WSL, por exemplo). Instale o pacote do sistema, ou acrescente o caminho ao `~/.profile` do usuário.

Deploy

`kamal config` reclama de variável faltando

Você esqueceu o `source .env`. Ele diz exatamente qual variável.

```
source .env && bundle exec kamal config
```

`denied` ou `unauthorized` ao empurrar a imagem para o ghcr.io

O `KAMAL_REGISTRY_PASSWORD` tem que ser um PAT (classic) com `write:packages` e `read:packages`. Um token de granularidade fina (*fine-grained*) não serve para o ghcr.io.

E o `GHCR_USER` é o seu usuário do GitHub, em minúsculas: o registry não aceita maiúscula no nome da imagem.

`exec format error` ao subir o container

Arquitetura errada: você buildou para `amd64` e a VM é `arm64` (ou o contrário).

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'dpkg --print-architecture'
```

Ajuste `SERVER_ARCH` no `.env`, `source .env` de novo, e refaça o deploy.

Kamal: `Docker is not installed`

O primeiro deploy precisa ser `setup`, não `deploy`:

```
source .env && bundle exec kamal setup
```

A aplicação sobe mas não acha o banco

`kamal deploy` **não** sobe accessories.

```
kamal accessory boot db
```

Depois de mudar env de accessory, `boot` não basta: é `kamal accessory reboot db`.

`curl` diz `self signed certificate`

Isso é o esperado, e é o exercício G da apostila. O túnel TLS subiu; o que faltou foi confiança.

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

curl diz **Could not resolve host: seunome.test**

Falta a linha no `/etc/hosts`:

```
echo "SEU_IP seunome.test" | sudo tee -a /etc/hosts
getent hosts seunome.test
```

No Windows, o navegador consulta `C:\Windows\System32\drivers\etc\hosts`, não o do WSL2.

curl conecta mas devolve 404 do proxy

O `APP_HOST` do `.env` tem que ser **exatamente** o nome que você está chamando. O `kamal-proxy` roteia pelo cabeçalho `Host`: se o certificado é para `seunome.test` e o `APP_HOST` está `outro.test`, ele não entrega a ninguém.

curl dá **Connection refused** na 443

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'docker ps && sudo ufw status'
```

O `kamal-proxy` está na lista? A 443 está liberada no `ufw`?

SMTP dá timeout na VM

Da VM local, a saída costuma funcionar: se não funcionar, é o firewall da sua rede (algumas redes universitárias bloqueiam a 587).

Numa VPS é mais comum: vários provedores de nuvem bloqueiam a saída na porta 25, e alguns na 587. Teste a alternativa do seu provedor de e-mail (o Resend aceita 2587) e ajuste `SMTP_PORT` no `env.clear`.

O deploy passa mas o site responde 500

```
source .env && kamal app logs -f
```

Suspeitos comuns: migration pendente (`kamal app exec "bin/rails db:migrate"`), `RAILS_MASTER_KEY` errada, ou um `ENV.fetch` sem default para uma variável que ninguém cadastrou.

Preciso voltar atrás agora

```
source .env && kamal rollback
```

Volta para a versão anterior da imagem, que ainda está na VM.

Apêndice: nuvem (Azure e Cloudflare)

Estes só acontecem no percurso de `apendice-azure.md`.

`NotAvailableForSubscription` ao criar a VM

O tamanho escolhido não está habilitado para a sua assinatura naquela região.

Assinaturas → **Uso + cotas** → **filtre por Compute** e escolha outra região ou outro tamanho. Pedir aumento de cota **não** resolve quando a oferta não está habilitada.

SSH dá timeout na VM da Azure

Quase sempre é o NSG:

1. o seu IP mudou? `curl -4 ifconfig.me` e compare com a regra;
2. a regra libera a porta 22, protocolo TCP, direção Entrada?
3. a prioridade da regra de permissão é **menor** que a de alguma negação?

Perdi o acesso depois de mexer no sshd, e não tenho sessão aberta

Sobrou o **Serial Console** do portal da Azure (menu da VM → Suporte + solução de problemas → Console Serial), que não passa pelo SSH.

OIDC: `AADSTS700213`

O *subject* da credencial federada não bate com o que o GitHub emite. Ele precisa ser, caractere por caractere:

```
repo:SEU-USUARIO/SEU-REPO:ref:refs/heads/main
```

Confira maiúsculas, o nome do repositório e a branch.

`az: command not found` ou falha transitória do Azure CLI

O `azure/login` às vezes falha por instabilidade. Rode o workflow de novo antes de investigar.

A regra temporária ficou no NSG

O passo de remoção tem `if: always()`, mas se o job for cancelado à força a regra pode sobrar. Apague na mão:

```
az network nsg rule delete \  
  --resource-group SEU_RG --nsg-name SEU_NSG \  
  --name allow-github-actions-deploy-ssh
```

E confira depois de todo deploy que falhou de forma estranha.

Cloudflare 521 / 522

O Cloudflare não conseguiu falar com o seu servidor.

- a porta 443 está aberta no NSG?
- `docker ps` na VM mostra o `kamal-proxy` rodando?
- o registro A aponta para o IP certo?

Cloudflare 525 / 526

Falha no handshake TLS entre Cloudflare e a sua VM.

- **525**: o proxy não apresentou certificado. Os secrets `KAMAL_PROXY_SSL_*` estão cadastrados?
- **526**: o certificado é inválido para o modo Full (strict). Ele cobre o hostname exato? Um wildcard `*.capacita.exemplo.tech` cobre `seunome.capacita.exemplo.tech`, mas **não** cobre `x.y.capacita.exemplo.tech`.

Let's Encrypt falha atrás do Cloudflare

Esperado. Com o DNS proxied, o desafio HTTP-01 não chega como o Let's Encrypt espera. Use o certificado Origin CA: é a razão de ele existir.